LLARS: Enabling Domain Experts & Developers Collaboration for LLM Prompting, Generation, and Evaluation

Philipp Steigerwald[1], **Mara Stieler**[2], **Jennifer Burghardt**[2], **Eric Rudolph**[1], **Jens Albrecht**[1]
Technische Hochschule Nürnberg Georg Simon Ohm, Nürnberg, Germany
[1]Faculty of Computer Science, Center for Artificial Intelligence (KIZ)
[2]Faculty of Social Sciences, Institute for E-Counselling
{philipp.steigerwald, mara.stieler, jennifer.burghardt, eric.rudolph, jens.albrecht}@th-nuernberg.de

Abstract

We demonstrate LLARS, an open-source platform that bridges the gap between domain experts and developers for building LLM-based assistants. LLARS integrates three modules into an end-to-end pipeline: **Collaborative Prompt Engineering** for real-time co-authoring with version control and instant LLM testing, **Batch Generation** for systematic output production across prompts × models × data with cost control, and **Evaluation** for human and LLM assessors to rate, rank, label, or compare outputs with live inter-rater agreement metrics. Interviews with six domain experts and three developers in psychosocial counselling confirmed that LLARS eliminates the "translation work" between disciplines and enables domain experts to directly co-author prompts and conduct evaluations independently.

Demo: https://llars-project.github.io/demo

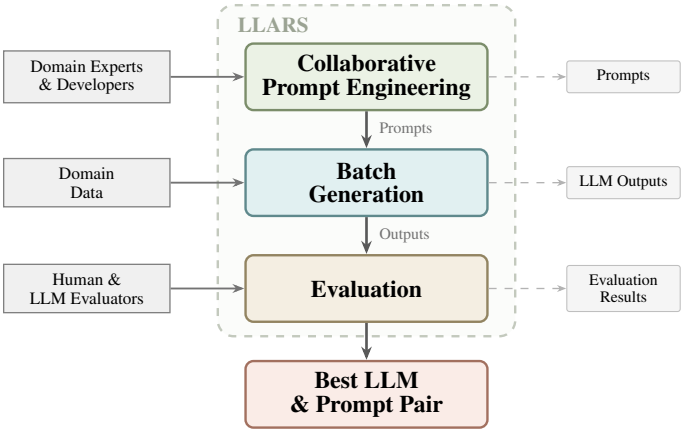

Figure 1: LLARS pipeline: domain experts and developers collaboratively develop prompts, generate outputs across LLMs, and evaluate results with human and LLM evaluators. Each stage supports export; the pipeline yields a validated model–prompt combination.

1 Introduction

Organizations building LLM-based assistants for sensitive domains [?; ?]—psychosocial counseling [?], legal analysis [?], medical documentation [?]—must write effective prompts, select suitable models, and rigorously evaluate outputs, requiring both domain expertise and technical skill [?; ?]. Yet these two groups typically work in isolation: prompts are iterated in shared documents without version control, and no single tool supports the full cycle from prompt development through systematic evaluation.

We present LLARS (LLM Assisted Research System), an open-source platform that closes this gap with three integrated modules (Figure 1). *Collaborative Prompt Engineering* lets domain experts and developers co-author prompts in a shared real-time editor with version control and instant LLM testing. *Batch Generation* systematically produces the Cartesian product of prompts × models × data items with cost estimation and budget control. *Evaluation* lets human and LLM evaluators rate, rank, label, or compare outputs with live agreement metrics. Each module supports JSON/CSV export and connects to any OpenAI-compatible provider.

Table 1 surveys twelve existing platforms along these dimensions. Agenta [?] combines prompt versioning with LLM-as-judge scoring and provides A/B deployment testing, but lacks real-time co-editing and structured human evaluation with inter-rater agreement metrics. Phoenix [?], Langfuse [?], and W&B Weave [?] provide observability dashboards with dataset experiments and per-output feedback but focus on tracing and debugging rather than systematic batch generation or coordinated multi-evaluator campaigns. ChainForge [?] offers a visual node-based interface for multi-model comparison with template variables but omits human evaluation entirely. Label Studio [?] and Argilla [?] offer structured human annotation—Label Studio computes Krippendorff's α, Argilla provides agreement analytics via its API—but require externally generated outputs, disconnecting evaluation from prompt development. LangSmith [?], Braintrust [?], Maxim [?], and Vellum [?] link prompt management to evaluation with dataset-level scoring but are closed-source, lack collaborative editing, and omit batch generation as a pipeline stage. LLARS is an open-source platform to unify collaborative prompt engineering, Cartesian-product batch generation, and hybrid human–LLM evaluation with agreement analytics, enabling developers and domain experts to work together from prompt development through rigorous assessment.

Table 1: Comparison of GUI-first tools for prompt engineering, batch generation, and hybrid LLM output evaluation. ● = native support; ◐ = partial; ○ = not supported. **Collab.** = real-time collaborative prompt co-editing; **Batch** = systematic generation over prompts × models × data; **H-Eval** = structured human evaluation campaigns; **LLM-E** = LLM-as-evaluator; **IRR** = inter-rater reliability metrics; **E2E** = integrated end-to-end pipeline.

Tool	OSS	Collab.	Batch	H-Eval	LLM-E	IRR	E2E
LLARS	●	●	●	●	●	●	●
Agenta [?]	●	◐	◐	◐	●	○	◐
ChainForge [?]	●	○	●	○	◐	○	◐
Phoenix [?]	●	○	◐	◐	●	○	○
Langfuse [?]	●	○	◐	◐	●	◐	○
W&B Weave [?]	●	○	◐	◐	●	○	○
Label Studio [?]	●	○	○	●	◐	◐	○
Argilla [?]	●	○	○	●	◐	◐	○
LangSmith [?]	○	○	◐	◐	●	○	◐
Braintrust [?]	○	○	◐	◐	●	○	◐
Maxim [?]	○	○	◐	◐	●	○	◐
Vellum [?]	○	○	◐	◐	●	○	◐

2 Platform Overview

LLARS runs as a containerized web application for textual data where teams register their own commercial or self-hosted OpenAI-compatible LLM providers; registered models then appear across all three modules: collaborative prompt engineering, batch generation, and evaluation.

2.1 Collaborative Prompt Engineering

The prompt engineering module (Figure 2) provides a shared workspace where domain experts and developers co-author prompts in real time using a collaborative editor built on Yjs [?]; every keystroke is instantly visible to all connected users, and online presence indicators show who is currently editing.

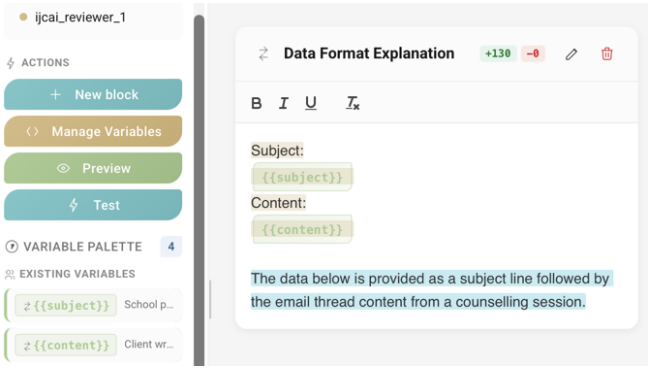

Figure 2: Collaborative prompt editor with a data-format explanation block containing template variables {{subject}} and {{content}}. The sidebar (left) shows actions for block management, variable management, preview, and LLM testing; the Variable Palette lists all extracted variables with sample values.

A prompt is composed of ordered blocks—each representing one part of the prompt, such as a data format description or an instruction for the desired output style—that can be freely added, reordered, or removed. One block may optionally be marked as a system block, which becomes the LLM's system prompt; all remaining blocks are concatenated into the user prompt. Each block maintains an independent version history that tracks insertions and deletions per save (visible as `+130 / −0` in Figure 2), enabling diff comparison and rollback to any prior revision without affecting other blocks.

Blocks may contain template variables written as {{variable_name}}, which act as placeholders for external data that would otherwise clutter the prompt—an entire email thread, for instance, can be captured in a single {{content}} variable, keeping the surrounding instructions concise and letting authors focus on prompt design rather than data formatting. The platform automatically extracts all variables into a shared Variable Palette (Figure 2, bottom-left), where each variable can be annotated with a sample value (e.g. {{subject}} = *"School problems at home"*). These samples let collaborators preview the assembled prompt and serve as test inputs: while editing continues, any team member can send the current prompt—with samples substituted—to a selected LLM and receive a streamed response, enabling one collaborator to test a self-hosted model while another simultaneously queries a commercial API. Although designed as the first pipeline stage, the editor also functions standalone: prompts can be exported as JSON (including blocks, variables, and version metadata) for external use, and existing prompts can be imported to continue editing collaboratively. Once a prompt is finalized, it passes directly into batch generation, where the same template variables are systematically filled from dataset columns, scaling a single prompt to hundreds of outputs.

2.2 Batch Generation

While the prompt editor allows rapid testing with sample values, a single example cannot validate whether a prompt generalizes across diverse inputs. Batch generation addresses this: users select from available prompts—self-authored or shared by collaborators—and any registered model to apply to their data, producing the Cartesian product of prompts × models × data items. A generation matrix previews this combination and estimates cost based on

model pricing and projected token counts; optional budget caps pause jobs that exceed a threshold. A wizard offers three data source modes, each suited to a different stage of development: upload data (provide new CSV or JSON input), previous generation (reuse data from a completed batch job), and prompt only (generate without source data).

In upload-data mode, users provide a CSV or JSON file whose columns map to the prompt's template variables—for example, 50 counselling email threads as JSON, each entry containing `subject` and `content` fields matching the variables defined during prompt engineering. Combined with two prompts and two models, LLARS produces $50 \times 2 \times 2 = 200$ outputs.

Previous-generation mode works identically but reuses the data from a completed batch job, so that new prompts or additional models can be tested on exactly the same dataset without re-uploading.

Prompt-only mode sends selected prompts to selected models without uploading data; if the prompt contains variables, the sample values from prompt engineering are used.

Results stream in real time, and every output records its full provenance—source item, prompt version, model, parameters, token count, and cost—exportable as CSV or JSON. This provenance is central to the pipeline: when outputs reach evaluation, assessors see only the generated text without any indication of which model or prompt produced it, and the platform maps their judgments back to specific model–prompt combinations, directly informing selection. Completed batches transfer into evaluation with a single click, preserving all attribution.

2.3 Evaluation

The evaluation module brings human and LLM evaluators together in a single framework. Four general evaluation types are available (Figure 3): *rating* with configurable multi-dimensional Likert scales, *ranking* into ordered buckets, *labeling* for binary or multi-class classification, and pairwise *comparison*. The type system is extensible: for the counselling domain, two additional types were added—*mail rating* for assessing email thread quality and *authenticity* for distinguishing human-written from LLM-generated text (Figure 3). An AI-assisted Scenario Wizard analyzes uploaded data to recommend the most suitable type; scenarios can also be created directly from batch generation outputs, preserving model attribution.

Scenario owners control how items are distributed—all items to every evaluator for full inter-rater analysis, or configurable subsets for faster throughput. Where most tools limit LLMs to pairwise judging [?], LLARS treats them as full evaluation participants that rate, rank, label, and compare alongside humans using customizable type-specific prompts. Agreement metrics update live—Krippendorff's α, Cohen's and Fleiss' κ, Kendall's τ, and ICC—filterable by evaluator pairing (human–human, LLM–LLM, human–LLM). All data, metrics, and configurations are exportable as JSON or CSV.

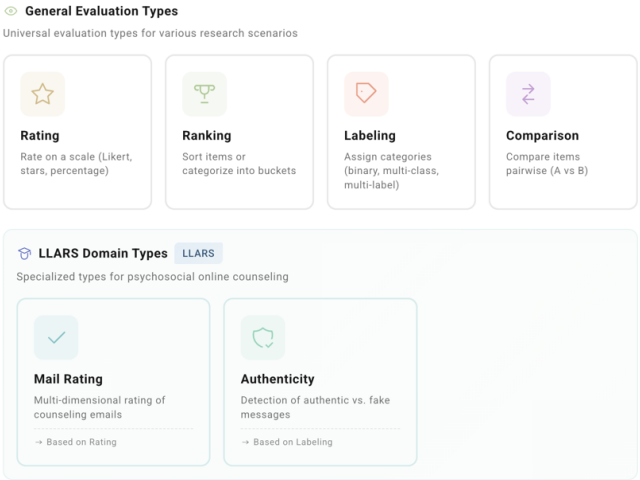

Figure 3: AI-assisted Scenario Wizard showing the task selection step with four general evaluation types and two domain-specific types for psychosocial counseling. The five-step wizard analyzes uploaded data and recommends the most suitable evaluation type.

3 Deployment and User Study

LLARS is actively used in psychosocial counselling research: the Virtual Client project [?] uses it to develop prompts for simulated client interactions, and CAIA [?], a co-designed AI assistant for online counselling, relies on it for prompt development and evaluation. A full end-to-end case study on automated subject line generation [?] refined prompts collaboratively, produced 253 subject lines across 11 LLMs via batch generation, and had five professionals together with an LLM evaluator rate them (1,518 assessments)—determining how small a model could be while still producing meaningful results.

Semi-structured interviews with six domain experts and three developers confirmed that consolidating the full pipeline in a single platform saves considerable time. Regarding *prompt engineering*, domain experts found that the collaborative editor made prompt authoring straightforward—particularly when working with self-hosted models, where no external tooling or playground exists. For *batch generation*, developers valued that prompts, data, and outputs remain in one place, eliminating the manual export–import cycles between separate tools that previously slowed iteration. The *evaluation* module was rated as immediately understandable by all domain experts: clear progress indicators show which items have been assessed and which remain, letting evaluators resume across sessions without losing context. Across all three modules, both groups emphasised that co-locating the entire workflow removed the "translation work" between disciplines and created the experience of "truly working together on the same task."

4 Limitations, Demo, and Future Work

LLARS has so far been deployed in a single domain with a small user base, so broader studies are needed to assess generalisability. The quality of LLM evaluator judgments depends

on careful prompt design, and the current evaluation relies on qualitative interviews rather than a controlled study. While the variable system helps prevent overfitting during prompt development, practitioners noted that prompts optimised on test cases can still underperform on unseen real-world data—a limitation inherent to any iterative prompt engineering process.

Our live demo walks attendees through the full pipeline, from collaborative prompt editing through batch generation to evaluation with live metrics. Future work targets a controlled user study comparing LLARS against baseline workflows (shared documents and spreadsheets) and cross-domain validation in legal and medical settings. LLARS is open-source and under active development; it continues to be deployed in ongoing projects to bring domain experts and developers together so that diverse fields beyond counselling can benefit from the current advances in AI.